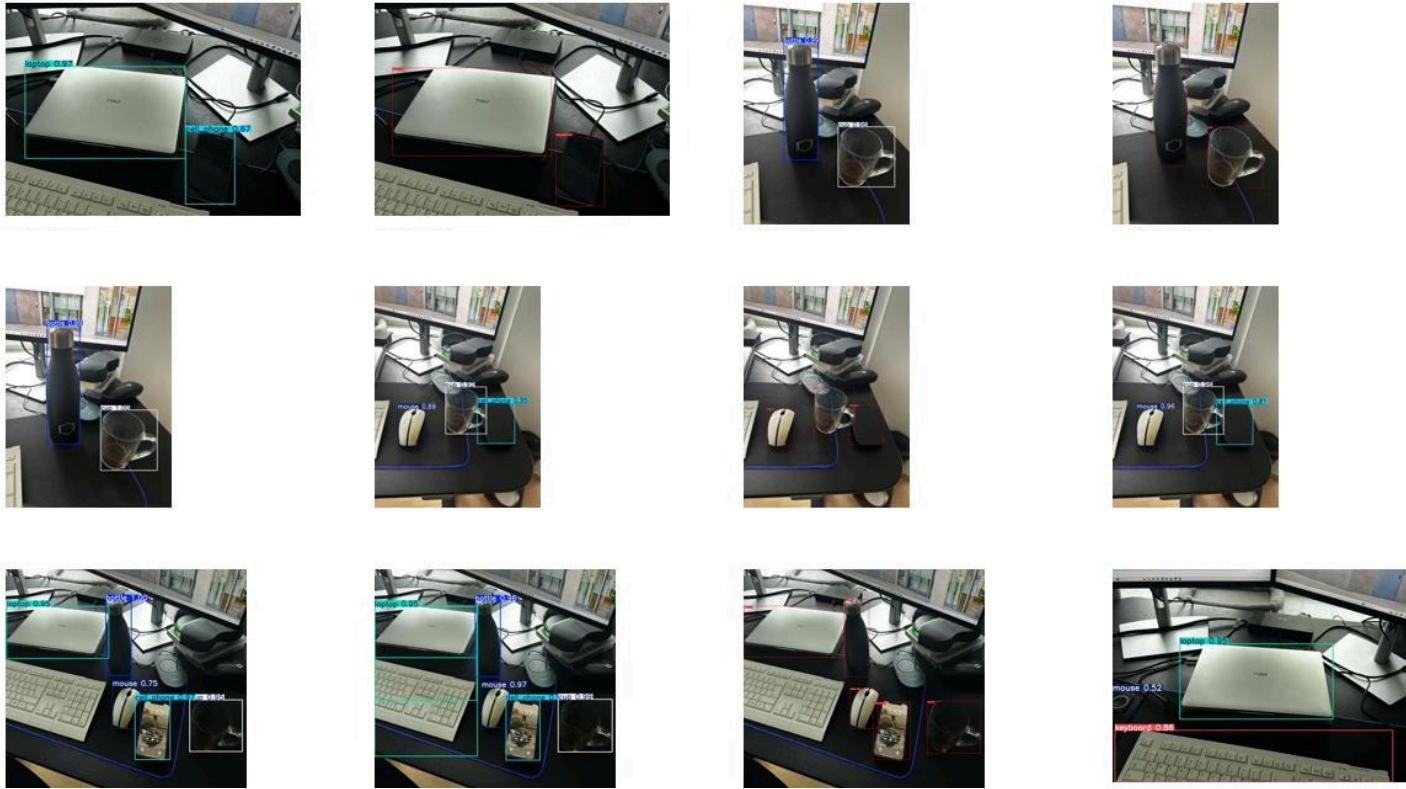


Problem

Many object detectors are trained on general image datasets, but real desk scenes are cluttered, partially occluded and contain small objects. The task is to detect and localize five relevant desk objects: **bottle, cell phone, cup, laptop** and **mouse**.

Custom desk scenes with manually annotated target objects.



Related Work / Motivation

Object detection can be approached with one-stage and two-stage architectures.

YOLO predicts objects in one forward pass and is commonly used for efficient real-time detection. **Faster R-CNN** first proposes object regions and then classifies them, which makes it a useful accuracy-focused comparison.

- Transfer learning reduces training effort on small custom datasets.
- A second method makes the project stronger and easier to defend.
- Comparing architectures shows both speed-oriented and region-based detection behavior.

Your Approach / Solution

The complete workflow was implemented from data collection to model comparison. A custom dataset was created in Roboflow and exported in YOLOv8 format. YOLOv8n was fine-tuned in three versions, then Faster R-CNN was trained on the final clean dataset.

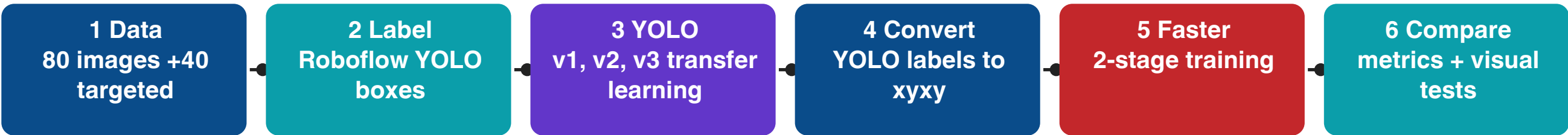
Dataset	Images	Augmentation	Purpose
v1	80	none	first YOLO run
v2	120	brightness	targeted improvement
v3	120	none	final clean set

Key engineering steps:

- Manual annotations for 5 classes only
- YOLO label format converted to xyxy boxes for Faster R-CNN
- Common 5-image visual test set used for both methods
- Final comparison: YOLOv8n v3 vs Faster R-CNN v1

Method / Pipeline / Algorithm / Process

The project compares two object detection strategies on the same custom desk dataset and uses a reproducible training and evaluation workflow.



Method 1: YOLOv8n

One-stage detector. It predicts boxes and classes in a single forward pass. Trained for 50 epochs with image size 640 and batch size 8. Three dataset versions were tested.

Final model: **YOLOv8n v3**

Method 2: Faster R-CNN

Two-stage detector. A Region Proposal Network suggests object regions, then a second stage classifies and refines boxes. Trained for 10 epochs on Dataset v3.

Final model: **Faster R-CNN v1**

Dataset and label handling

Roboflow exported labels in YOLO format: **class_id x_center y_center width height**. Faster R-CNN requires absolute boxes: **x_min y_min x_max y_max**. A custom PyTorch dataset loader converts the boxes and shifts labels by +1 because label 0 is reserved for background.

YOLO ID	Class	Faster R-CNN Label
0	bottle	1
1	cell_phone	2
2	cup	3
3	laptop	4
4	mouse	5

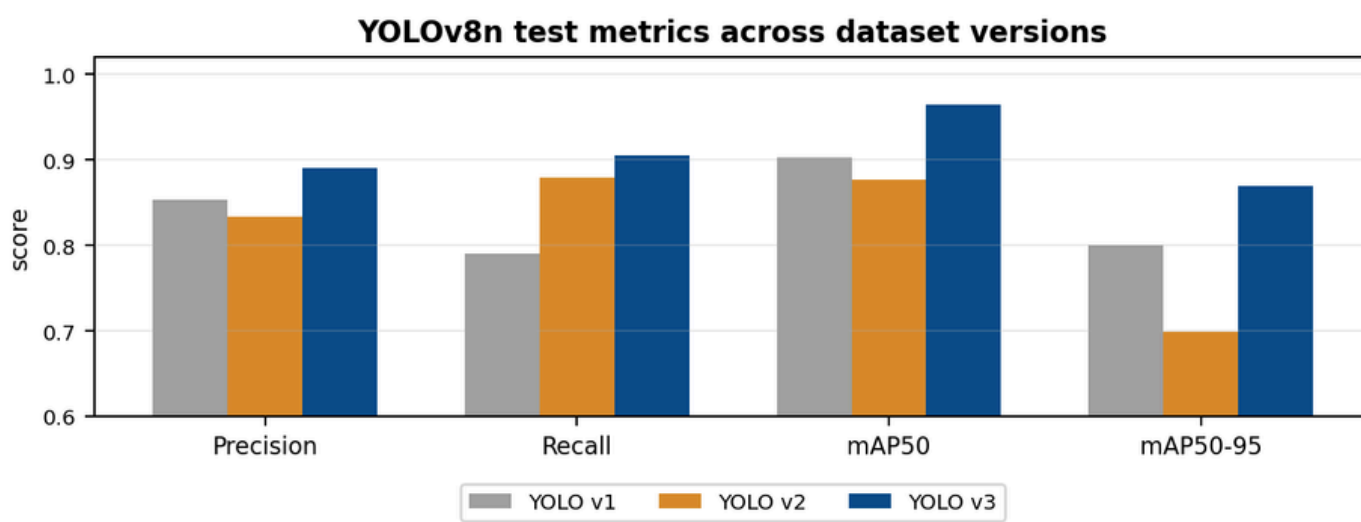
The same Dataset v3 was used for the final YOLO and Faster R-CNN comparison.

Results

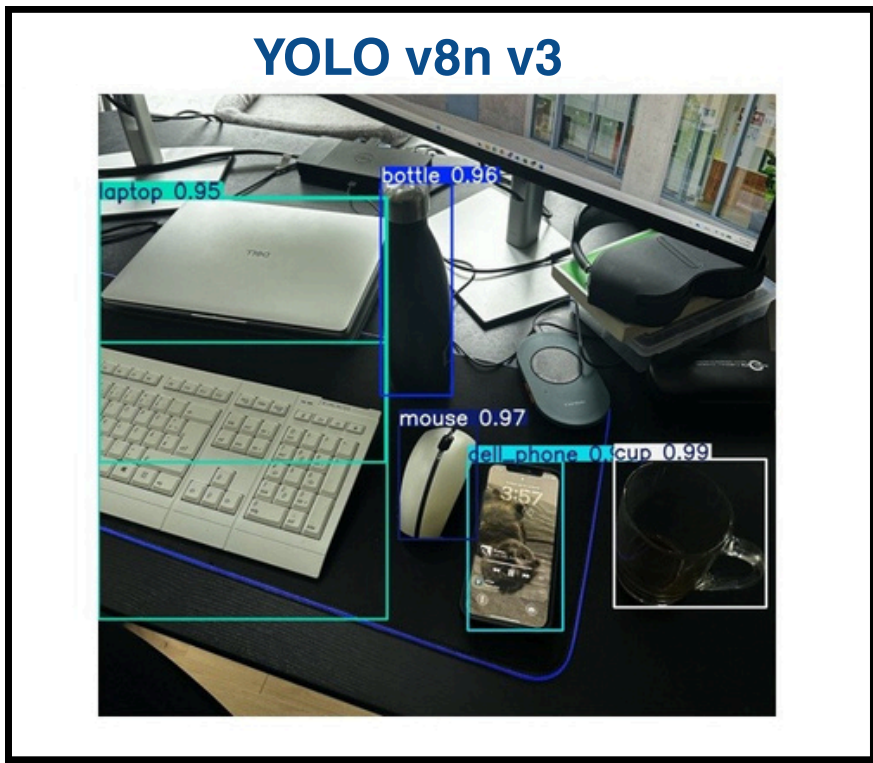
The final evaluation combines YOLO metrics, Faster R-CNN training loss and visual predictions on the same five example images.

YOLO Version	Dataset	Precision	Recall	mAP50	mAP50-95
v1	80, no aug.	0.853	0.791	0.903	0.800
v2	120, brightness	0.834	0.879	0.877	0.699
v3	120, no aug.	0.890	0.906	0.965	0.870

Best YOLO: v3. It improved all main test metrics. Dataset v2 showed that brightness augmentation increased visual false positives.



Visual comparison on the same 5 test images



Discussion and conclusion: Dataset quality had the strongest impact on performance. Targeted new images improved cell phone and mouse detection. Removing brightness augmentation made YOLO v3 cleaner. Faster R-CNN produced very clean visual predictions. YOLO is the better real-time candidate, while Faster R-CNN is the stronger region-based comparison.

Aspect	YOLOv8n v3	Faster R-CNN v1
Architecture	one-stage	two-stage
Training data	Dataset v3	Dataset v3
Strength	fast and efficient	clean region-based boxes
Main output	mAP50 0.965, mAP50-95 0.870	loss 0.0622, strong visuals

References

- [1] Ultralytics YOLOv8 documentation and pretrained YOLOv8n weights.
- [2] Torchvision Faster R-CNN ResNet-50 FPN implementation.
- [3] Roboflow annotation and YOLOv8 dataset export workflow.
- [4] COCO pretrained model weights used for transfer learning.
- [5] Course material: Maschinelles Sehen, BHT Berlin.

Repository contents: source code, datasets, trained models, evaluation outputs, prediction images and project documentation.